CHI FELLOWSHIP | TECHNICAL REPORT

The Self-Aware Room

A Technical Report and Reproducible Reference for a Responsive, Projection-Mapped Environment in TouchDesigner

Gabriel A. Aguilar
Immersive Director and Creative Technologist

New York City College of Technology (CUNY City Tech)
Self-Aware Room Initiative
Faculty Advisor: Professor David Smith

Fellowship period documented: July 15 to August 13, 2026
Prepared as a reference for subsequent fellows

Abstract

The Self-Aware Room is a projection-mapped environment designed to sense the people within it and respond to them in real time. This report documents the technical foundation established for that environment over the course of the fellowship period from July 15 to August 13, 2026. The work described spans the media-delivery, projection-mapping, and inter-system messaging layers upon which the room's later interactive behavior depends. For each system, the report provides both an explanation of what was developed and a reproducible procedure for rebuilding it. The systems addressed include the selection of a software license tier for TouchDesigner, single- and multi-projector projection mapping through the KantanMapper palette, the transmission of Open Sound Control messages within a single machine and across a local network, and the control of projected output through external input. The document is intended to function as a reference for subsequent fellows, consolidating the working methods and the recurring points of failure encountered during the fellowship.

Keywords: *TouchDesigner, Projection Mapping, Open Sound Control, Immersive Environments, Real-Time Media, Show Control, Networked Systems, Creative Technology.*

Contents

1. Introduction

The Self-Aware Room is a projection-mapped physical space designed to sense its occupants and respond to them in real time. Its long-term objective is an environment in which the surfaces and objects of a room are driven by live media, and in which a sensing system and a digital model of the space remain in agreement with the events occurring within it. The work documented in this report occupies the layer beneath that objective, comprising the delivery of media to projectors, the messaging that allows independent machines to share a single state, and the procedures required to reconstruct each system reliably.

This report is organized by system rather than by date. Where the daily logs recorded the chronological progress of the fellowship, this document consolidates that progress into a reference organized around the components of the room. Each section states what a given system is and the role it serves within the larger environment, and then documents the procedure for reproducing it. Where a procedure contains a recurring point of failure, that failure is described within the text so that it may be anticipated rather than rediscovered.

The report is intended to be read in two ways. A reader new to the project may proceed through the sections in order, beginning with the software platform and the licensing decision and continuing through single- and multi-projector projection mapping before reaching the messaging and interaction layers. A reader already familiar with TouchDesigner may proceed directly to the sections concerning Open Sound Control, networked communication, and input-driven control. External resources referenced throughout are collected by topic in Appendix C.

The systems documented below constitute a foundation rather than a completed work. Each functioned individually, and by the conclusion of the fellowship they functioned together in a single end-to-end test. The work that remains is the development of that pipeline into a dependable system suitable for live performance, followed by the development of the room's aesthetic content.

2. The TouchDesigner Platform and Licensing

Every visual system in the project runs within TouchDesigner, and the selection of a license tier was therefore the first structural decision of the fellowship. The room was initially developed on Non-Commercial keys, which are provided at no cost and permit full use of the environment for learning and testing. The Non-Commercial tier imposes two limitations that the Self-Aware Room will exceed. The following analysis documents the tier comparison prepared for Professor Smith so that the reasoning is preserved.

2.1 License comparison

The following table compares the four TouchDesigner and TouchPlayer tiers on the features relevant to a live, multi-projector, sensor-driven installation. An X indicates that the feature is included at that tier.

Feature	Non-Comm.	Educational	Commercial	Pro
Includes all basic operators	X	X	X	X

Feature	Non-Comm.	Educational	Commercial	Pro
Usable in paying projects			X	X
Transferable keys (computer to computer)		X	X	X
Unlimited resolution	1280x1280 limited	X	X	X
Use Engine COMP in TouchDesigner	X	X	X	X
Use TouchEngine in other applications		X	X	X
Hide splash screen on startup		X	X	X
TouchPlayer hide interface overlay		X	X	X
Includes all Shared Memory operators		X	X	X
Ouster LIDAR device support		X	X	X
SICK LIDAR device support		X	X	X
Leuze ROD4 laser scanner support		X	X	X
Send and receive textures to DirectX apps		X	X	X
Blob Track TOP blobs tracked	unlimited	unlimited	unlimited	unlimited
Video Stream Out TOP (RTSP / RTMP / SRT)	X	X	X	X
Export H.264 / H.265 in realtime (Nvidia GPU)		X	X	X
NDI operators for streaming over LAN	X	X	X	X
Notch TOP for running Notch blocks		2x blocks	2x blocks	X
3rd party projector calibration (Vioso / Scalable)				X
MoSys camera tracking				X
Stype camera tracking				X
Ncam camera tracking				X
BlackTrax motion tracking				X
RenderStream support				X
Sync multiple processes or systems				X
Hardware frame-lock (Nvidia Gsync / AMD S400)				X
Create Private .toe files and components				X
Animation Sequencing (Clip Blender CHOPs)				X
Forum support	X	X	X	X
Pro support (direct email / phone)				X
Bug fixes in the specific build you use				X
TouchDesigner price	No charge	$300 USD	$600 USD	$2200 USD
TouchDesigner Floating Cloud price	N/A	$450 USD	$900 USD	$3300 USD
TouchPlayer price	No charge	N/A	$300 USD	$800 USD

License and feature data as published by Derivative. Prices are listed in USD and were current at the time of writing.

2.2 Analysis of the relevant tier

The majority of the capability restricted to the Pro tier is intended for large broadcast and virtual-production facilities and is not required by this project. The features relevant to the Self-Aware Room are located almost entirely within the Educational tier. Transferable keys are significant because the machine running the room will vary with location, operator, and equipment availability; the Educational tier permits a shared pool of licenses for the project as a whole, while the Non-Commercial tier binds a key to a single machine. The resolution limit is the more consequential restriction. The Non-Commercial tier caps output at 1280 by 1280 pixels, while the finished room targets a resolution of 6K to 8K, which renders the Non-Commercial tier unsuitable for the final installation regardless of other considerations. TouchEngine, which allows a TouchDesigner component to run within other applications, is the most direct path to integrating TouchDesigner with QLab, the project's media server, and is likewise available beginning at the Educational tier. The remaining features of interest, including texture sharing with DirectX applications, real-time hardware video capture, and LIDAR device support, are also resolved at the Educational tier.

The Educational tier is therefore the appropriate license for the project. It removes each limitation the room actually encounters, while the Pro tier addresses requirements the project does not have. The single Pro feature of potential value, third-party projector calibration through Vioso or Scalable Display, has manual alternatives. Should the project eventually exceed the Educational tier, the more appropriate response would likely be the development of a dedicated application encompassing the full pipeline rather than the purchase of the Pro tier.

3. Projection Mapping with KantanMapper

KantanMapper is the projection-mapping palette included within TouchDesigner. It allows a video source to be warped onto an arbitrary quadrilateral or mesh so that the projected image conforms to a physical surface that is not oriented as a flat plane facing the projector. Projection mapping is foundational to the Self-Aware Room, as an unmapped projection is limited to a rectangular field of light and cannot conform to the geometry of the space. The procedure below documents the construction of a single mapped output, which serves as the template for the multi-projector configurations described in Section 4.

3.1 Constructing a KantanMapper patch

The following procedure produces a single mapped output running live in its own window.

1. Create a new project from the File tab.
2. Import the KantanMapper container into the project. Open the Dialogues menu, open the Palette Browser, open the Mapping category, and drag KantanMapper into the work field.
3. Drag a video file into the work field to serve as the source to be mapped.
4. In the KantanMapper window settings, engage Open as Separate Window, which activates the live output.
5. Disable Display in the panel settings so that the editor interface is not included in the projected output.

6. Create an OUT TOP and connect the KantanMapper to it.
7. Zoom out to reach the outscope window.
8. In the Perform window settings, set Borders off, set Monitor to 1, and set Opening Size to Fill.
9. Connect each video source to its mapped area by dragging and dropping.
10. Begin playback. If no output appears, toggle Output in the window panel.

A recurring point of failure occurs at this stage. If the Open as Separate Window control or the Toggle Output control is not engaged, the patch appears correctly configured within the network editor while producing no visible output, and no error is generated to indicate the cause. When a completed mapping produces no projection, these two controls should be verified before any further troubleshooting is attempted.

3.2 Configuring the projector

A projector is required in order to test a mapping. The unit used for the first mapping was the Optoma Short-Throw Pocket LED, selected for its small form factor. The following procedure configures it.

1. Connect the projector to a power source rated for its 19V input.
2. Connect the HDMI cable from the computer to the projector.
3. Mount the projector on a tripod.
4. Position the projection so that it falls centered on the target surface.
5. Open the Display menu, reverse the image if the orientation is incorrect, focus the projector, and adjust keystone as required.

This unit provides no zoom and no corner adjustment, and framing therefore cannot be corrected through its menu after placement. The projector must be positioned so that the projection is correctly framed at the outset, after which only focus and keystone remain to be adjusted. The physical placement of a projector of this type must be planned in advance, as the software provides no means of compensating for it.

3.3 Result

The template and the projector together produced a multi-surface projection driven by several video sources and mapped onto multiple objects within the projector's field of view. This result served as a technical verification of the mapping method. The aesthetic development of projected content is addressed separately in Section 10.

4. Multi-Projector Configuration

A single projector demonstrates the mapping technique; the room requires several projectors operating together. The subsequent objective was the operation of three projectors from a single machine, each displaying independently mapped content at a stable frame rate.

4.1 Architecture

The recommended structure assigns each projector an independent KantanMapper container and canvas, each feeding a separate output window on a separate display. This separation keeps the projection paths distinct and avoids combining the outputs within a single mapper. Figure 1 shows three independent movie sources, each routed to its own KantanMapper container.

Figure 1. Three independent KantanMapper containers, one per projector. Each movie source feeds its own mapper and its own output.

4.2 Equipment

The three-projector configuration requires the following.

- A computer with at least three video outputs connected to the graphics card.
- A TouchDesigner license. The Non-Commercial key is sufficient for testing; the final resolution requires the Educational license, as established in Section 2.
- One HDMI cable and two HDMI-to-USB-C cables, matched to the available ports.
- Three projectors. The units used in the documented configuration are EL-265F projectors.
- Power distribution planned in advance.

4.3 Procedure

1. Establish the power configuration first. The projectors and the computer can operate from a single power strip, though an individual line for each unit is the preferred practice, as it prevents a single fault from interrupting the entire configuration.
2. Build a patch containing three separate KantanMapper containers, one per projector, each with its own canvas.
3. Route a distinct video source into each mapper and send each mapper to a separate output window assigned to a separate display.
4. Set every output window to Perform mode before operation.

When three projectors were first brought into operation, playback slowed substantially. The cause was that the output windows were running in Working mode. Setting each projection window to Perform mode resolved the bottleneck. Every output window in a multi-projector configuration should be set to Perform mode before operation, as this is the principal determinant of frame rate in this configuration.

4.4 Bypassing the mapper and integrating a media server

Where a projector addresses a flat surface that does not require warping, KantanMapper may be bypassed and the output placed directly through the Window COMP. Mapping complexity should not be introduced for a surface that does not require it. Separately, QLab supports Syphon, which permits TouchDesigner and QLab to exchange video locally without capture loss. This provides a direct path for QLab to operate as the show controller in front of TouchDesigner.

4.5 Systems documentation

The completed configuration was documented as a systems diagram so that the physical setup can be reconstructed without reference to the original cable runs. The diagram records both signal and power: USB-C to HDMI from the computer to each projector, IEC C13 power to each unit, and an Edison stinger to the computer's power supply.

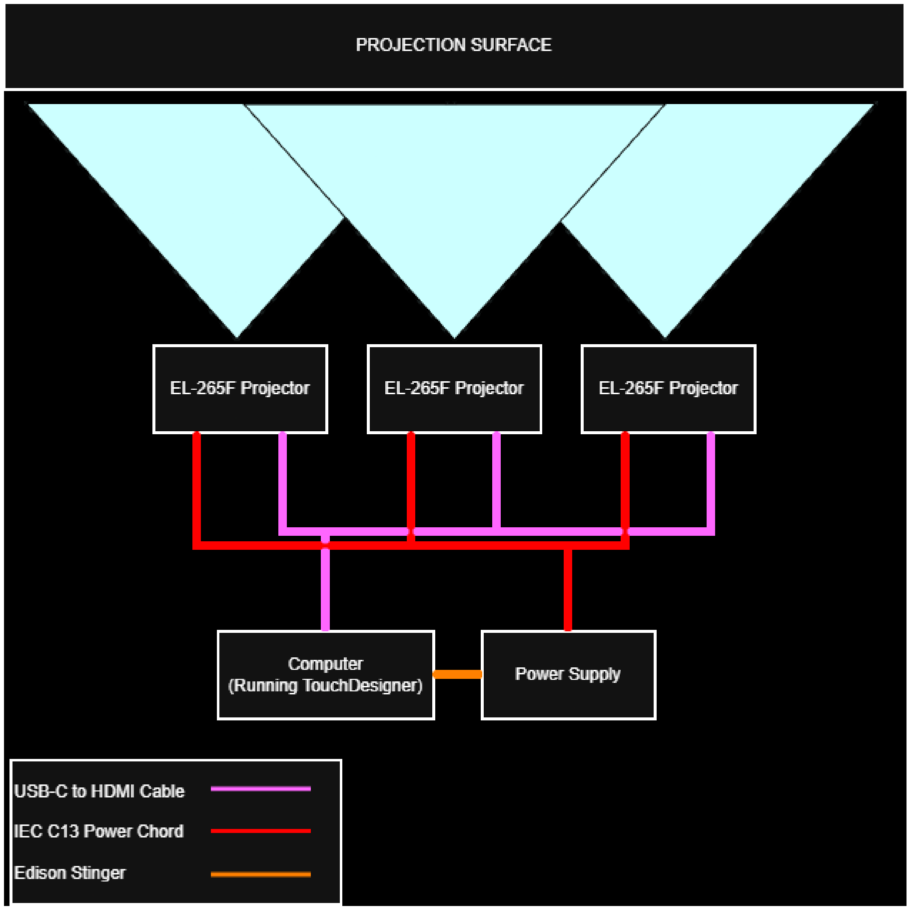

Figure 2. Systems diagram of the three-projector configuration. Three EL-265F projectors cover a shared projection surface. Magenta lines denote USB-C to HDMI signal, red lines denote IEC C13 power, and the orange line denotes the Edison stinger to the computer's power supply.

A single graphics card provides a limited number of outputs. Configurations exceeding approximately four projectors require multiple graphics cards, for which TouchDesigner provides specific guidance, referenced in Appendix C. This transition should be planned before the output limit of a single card is reached.

5. Show Control and Playback Software

The construction of visual content is distinct from its operation during a performance. Show control refers to the ordered triggering of content, the holding of states, and the recovery from error during a live event. Three tools were evaluated as candidates for the show-control layer above TouchDesigner.

CuryCue is a cue-based extension built upon TouchDesigner and directed at the management and triggering of content in live performance. Because it operates within TouchDesigner, it maintains cueing and rendering within a single environment. Isadora is notable for its extensive built-in human-tracking capabilities, which are directly relevant to an environment premised on responding to its occupants, though its adoption distributes the system across an additional application. QLab is the project's existing media server and is broadly familiar within live production; it connects to TouchDesigner either through TouchEngine, which requires the Educational license, or through Syphon, which is available at the current tier.

Each of these tools represents an external framework for the problem. A room required to perform cueing, sensing, rendering, and digital-model synchronization simultaneously may ultimately be better served by a dedicated application encompassing the entire pipeline than by the integration of several separate tools. This possibility is noted as a consideration for later development rather than an immediate recommendation.

6. Open Sound Control: Receiving Messages in TouchDesigner

Open Sound Control, hereafter OSC, is the messaging protocol underlying the project. It is a lightweight network protocol for transmitting named messages with associated values, and it is the mechanism through which the independent components of the room are intended to communicate: a controlling program indicating which asset to display, a sensor reporting a change within the room, and the digital model and the physical projection maintaining a shared state. The reliable reception of an OSC message within TouchDesigner is a prerequisite for this communication and was the starting point of the messaging work.

6.1 The OSC In DAT

OSC is received in TouchDesigner through an OSC In DAT. The operator is assigned a port on which to listen, and each incoming message is entered as a row containing its address and its arguments. The operator also exposes a Python callback, onReceiveOSC, which executes on the arrival of each message and is the point at which a received value is applied within the patch.

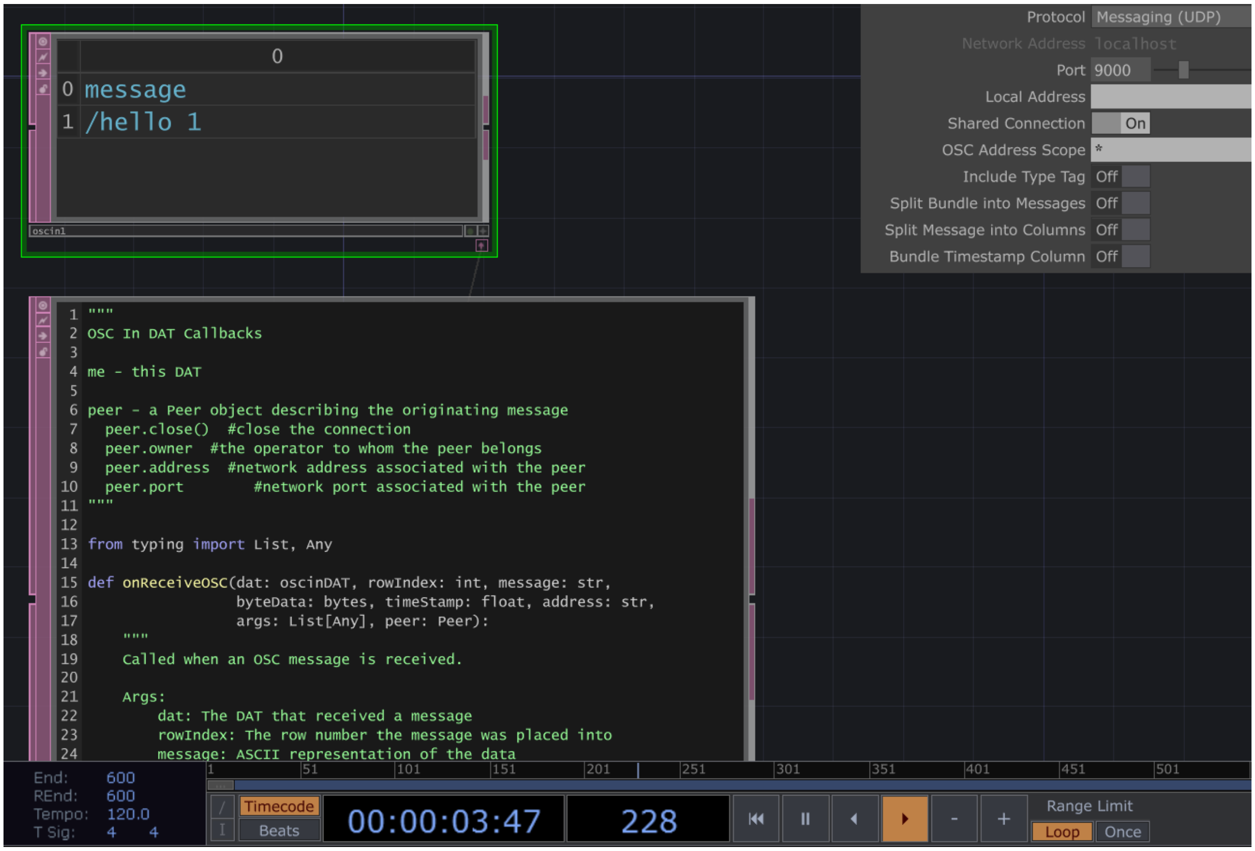

Figure 3. An OSC In DAT receiving the test message /hello with a value of 1 on UDP port 9000. The lower pane contains the onReceiveOSC callback, which executes on each incoming message.

6.2 Verifying reception

The following procedure confirms that a single message is received.

1. Add an OSC In DAT to the patch.
2. Set its Protocol to Messaging (UDP) and assign a Network Port on which to listen. Port 9000 was used for the first test.
3. From an OSC sender, transmit a named message such as /hello with a single integer value.
4. Confirm that the message appears as a new row in the DAT, showing its address and value. Reception is verified once the row appears.

6.3 Streaming an index

A single message verifies the configuration. The operational case is a value that changes continuously, and the principal pattern for this room is the transmission of an index, a single value indicating which asset should currently be displayed. Figure 4 shows TouchDesigner receiving an incrementing index on the address /test/index, which corresponds to the message form later used to select a video for playback.

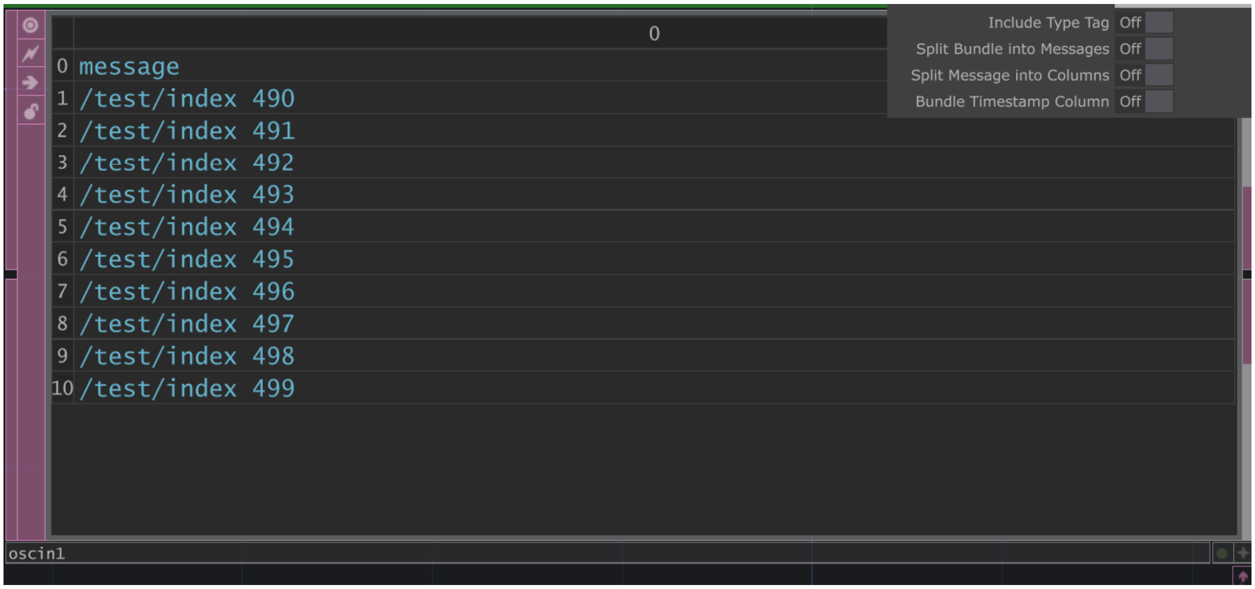

Figure 4. The OSC In DAT receiving a continuous stream of /test/index messages with incrementing values. This is the message form used to drive asset selection.

The Protokol utility, produced by Hexler, is a monitor that displays raw OSC traffic. It is useful for determining whether a communication failure originates with the sender or with the receiver, and is placed between the two to isolate the fault. The first working configuration was constructed using a sample project as a structural reference and was adapted until TouchDesigner displayed assets from an array of values transmitted by Sunima's Semantic Engine.

7. Networked Communication Between Machines

The reception of OSC on a single machine is a preliminary exercise. The room comprises several machines that must maintain agreement in real time, and the messages must therefore traverse a network. The majority of the difficulties encountered during this portion of the work concerned the network rather than the media.

7.1 Verifying connectivity

Before any application-level debugging, the machines must be confirmed to reach one another. The ping command transmits a small packet and awaits a reply; a result of zero percent packet loss indicates that the network path is sound and that any remaining fault lies above the network layer, in a firewall or in the application. Figure 5 shows a successful ping to 192.168.50.10.

```
C:\Users\ubiqu>ping 192.168.50.10

Pinging 192.168.50.10 with 32 bytes of data:
Reply from 192.168.50.10: bytes=32 time=4ms TTL=64
Reply from 192.168.50.10: bytes=32 time=2ms TTL=64
Reply from 192.168.50.10: bytes=32 time=2ms TTL=64
Reply from 192.168.50.10: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.50.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 2ms, Maximum = 4ms, Average = 2ms

C:\Users\ubiqu>
```

Figure 5. A successful ping to 192.168.50.10 with four replies and zero percent packet loss, confirming that the network path between the machines is sound.

7.2 Establishing the network

The configuration documented here connects two machines over a dedicated Ethernet link through a NETGEAR GS316EPP switch. The sending machine, a Mac running the Python program and the Semantic Engine, was assigned the static address 192.168.50.10. The receiving machine, a Windows computer running TouchDesigner, was assigned 192.168.50.20. No gateway or DNS was configured on the Ethernet adapters, so that the Ethernet link carries only the room's traffic while each machine retains an independent internet connection over Wi-Fi. The following procedure reproduces this configuration.

1. Connect the machines by Ethernet through a dedicated switch. A NETGEAR GS316EPP switch was used so that the room's traffic occupies its own local network rather than a shared or wireless connection.
2. Assign each machine a static address on the same subnet, and leave the gateway and DNS fields blank on the Ethernet adapters. In this configuration the sending machine was 192.168.50.10 and the receiving machine was 192.168.50.20.
3. In TouchDesigner, add an OSC In DAT, set its Protocol to Messaging (UDP), and set its Network Port to the agreed port, which was 9000. Leave the Local Address field blank so that the operator listens across all interfaces, including the Ethernet link.
4. Open the OSC port through the firewall on every machine that will send or receive, as described below.
5. Install any additional dependencies required by the sending program through the command terminal.

Firewall configuration is the most significant factor in establishing OSC communication over Ethernet. Both machines must permit traffic on the OSC port. On the Windows receiver this was resolved by allowing TouchDesigner through the Private network profile at the application level, and by adding an explicit inbound UDP rule for the listening port, scoped to the Private profile, within Windows Defender Firewall with Advanced Security. If the port is not opened, messages are discarded without notification: the sender registers no error, the receiver displays nothing, and the physical connection tests as sound throughout. When a ping succeeds but OSC does not arrive, the firewall is the most probable cause.

The configuration was verified in stages. A bidirectional ping confirmed the network path; a single /hello message confirmed that TouchDesigner received and parsed OSC correctly; and a burst of five hundred messages transmitted without loss confirmed that the link remained stable under continuous load. The listening port can be confirmed at the operating-system level with the netstat command, which reports whether the port is open and bound.

7.3 A fault resembling a network failure

The most persistent fault encountered during the integrated test was not, in fact, located in the network. The Python program was being executed from an incorrect repository. The program appeared to transmit correctly and its output appeared plausible, and a considerable period was spent examining the network before the cause was identified. The executing code was not the code under revision. Once the incorrect execution path was corrected, communication proceeded. The general principle is that the identity of the executing file should be verified before a network fault is assumed.

The first successful transmission between machines was conducted with Sunima. Commands transmitted from her machine arrived on the receiving machine within TouchDesigner and were interpreted correctly. As device-to-device messaging is the basis for all subsequent interaction within the room, this result constituted the principal milestone of the messaging work.

8. Driving Projection Output with Open Sound Control

With reliable message reception established across the network, the next requirement was that an incoming OSC value determine which asset is projected. This is the junction between the messaging layer and the visual layer. The design principle adopted at this stage is that TouchDesigner operates as a passive receiver: it receives control data over OSC, selects an asset on the basis of that data, and routes the resulting texture to the projector, without originating control logic of its own. The configuration described below uses stock operators throughout and can be constructed without custom scripting.

8.1 Network architecture

The path from the sending program to the projector proceeds through a fixed sequence of operators. An OSC In operator receives the message; a Select operator isolates the relevant value; that value drives the Index parameter of a Switch TOP; the Switch TOP selects among a bank of pre-loaded assets; and the selected texture passes through KantanMapper to a Window COMP configured for the projector. Figure 6 records this architecture.

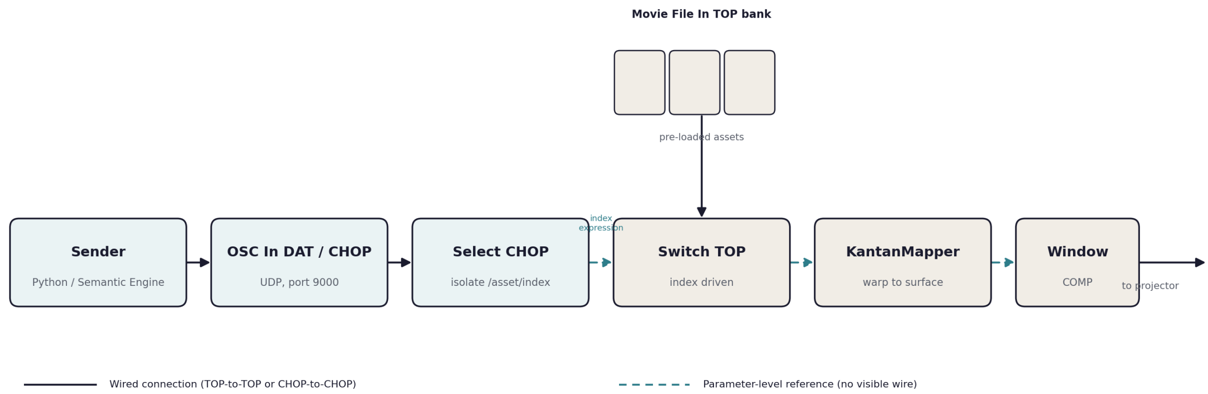

Figure 6. The OSC-driven output architecture. A Select operator isolates the incoming index and drives the Index parameter of a Switch TOP, which selects among a bank of pre-loaded Movie File In sources. Solid connections denote wired operator inputs; dashed connections denote parameter-level references, which carry no visible wire.

8.2 Asset selection: two methods

Two methods resolve an incoming value to an asset. The first, appropriate for a bounded and known set of assets, is a Switch TOP bank. Each asset is loaded into its own Movie File In TOP, all of which are connected to a single Switch TOP, and the switch's Index parameter selects among them. This method is the most stable in real time, because every asset is resident in memory and a change of index incurs almost no cost. It is the recommended default and is the method used in the constructed patches. Figure 7 shows a realized configuration of this kind.

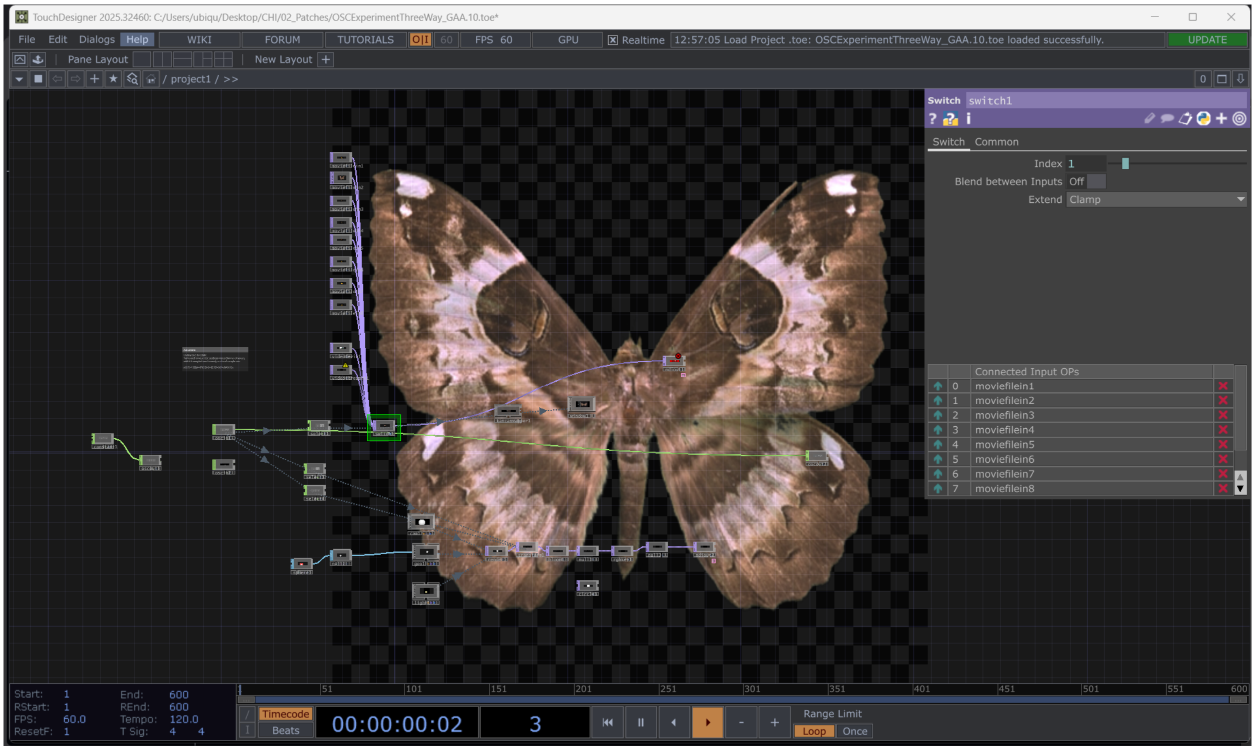

Figure 7. The OSCExperimentThreeWay patch, a realized Switch TOP bank. Eight Movie File In sources are connected to a single Switch TOP; the connected input list at right shows moviefilein1 through moviefilein8. The Index parameter, driven by OSC, selects which source reaches the output.

The second method, appropriate for a large or variable library, resolves a file path dynamically. A Folder DAT indexes every file in the asset directory by name, path, and index; the incoming OSC string is matched against that index using a Lookup or a Select DAT; and the resulting path is written to the file parameter of a single Movie File In TOP through a parameter expression. This method conserves memory but is more likely to interrupt playback, because a new file is opened on each change rather than selected from resident sources.

8.3 Mapping the incoming value to the index

The form of the mapping depends on whether the sending program transmits a numeric index or a string. When the value is a numeric index, an OSC In CHOP presents each OSC address as a channel, a Select CHOP isolates the relevant channel, and the channel's value is applied to the Switch TOP's Index parameter through a parameter expression of the form op('select1')['chan1']. This routes the value without any scripting. When the value is a string, an OSC In DAT presents the message as address and value rows, a Select DAT isolates the relevant rows, and the string is matched against the Folder DAT to yield an index or a path. Where the value is instead applied within the OSC In DAT's onReceiveOSC callback, the same result is achieved through the callback rather than a parameter expression; the parameter-expression method is preferred where a script-free patch is required.

A distinction must be drawn between a continuous value and a momentary trigger. If the sending program transmits a single pulse per event rather than a persistent value, the pulse is converted to a held value by a Trigger CHOP or a Count CHOP, so that the Switch TOP reads a stable index between events.

8.4 Connections within TouchDesigner

Not every relationship in this network is a visible wire, and the distinction is a frequent source of confusion. Connections that carry pixel data between two TOPs, such as the Movie File In sources feeding the Switch TOP, are wired directly, and connections between two CHOPs are likewise wired. The value that drives the Switch TOP's Index parameter, by contrast, is a parameter-level reference entered in the parameter field rather than a wire, and the same is true of the relationships between the Switch TOP and KantanMapper and between KantanMapper and the Window COMP. KantanMapper in particular does not accept a wired input for its source texture; the texture is assigned within KantanMapper's own floating editor. A patch that appears to be missing connections may therefore be correctly configured through parameter references.

8.5 Maintaining frame stability during rapid switching

Several measures preserve a stable frame rate when the index changes frequently. All candidate assets should be pre-loaded, as in the Switch TOP bank, so that a change of index is a selection rather than a file open. The sources feeding the switch should share a common resolution and format, because a mismatch forces an internal reformat on every change, which is a frequent and concealed cause of stalls. Rapid or overlapping triggers should be smoothed, or debounced, by a Lag CHOP or a Filter CHOP with a short window, or gated by a rate-limiting Logic CHOP, so that the output does not switch faster than it can settle. With all inputs pre-loaded, the Switch TOP cooks only the active branch by default, which keeps the load low even with many resident assets. For still images, a Cache TOP placed after the switch holds the last frame across any gap in the signal and prevents a blank flash between assets. The output window should be set to Perform mode, as established in Section 4.

8.6 Verification and debugging

The messaging path can be verified in isolation, before the sending program is involved, by constructing a loopback within TouchDesigner. A Constant CHOP feeds a value through a Null CHOP into an OSC Out CHOP, which transmits to the OSC In operator on the same machine, from which the value proceeds through the Select operator to the Switch TOP. This confirms the internal chain independently of the network.

A specific error recurs at this stage. The Switch TOP reports that a float argument received a value of type NoneType. The cause is that the Select operator returns no value, which occurs when no matching OSC channel has yet arrived, or when the channel name expected by the Select operator does not match the channel name generated from the OSC address. The resolution is to confirm that the OSC address and the expected channel name correspond, and to ensure that a value is present before the Switch TOP reads it.

An OSC In DAT may be added in parallel with the CHOP chain as a debugging tap, so that the raw incoming messages can be inspected in table form without disturbing the operators that drive the output. At the operating-system level, the netstat command confirms whether the listening port is open and bound, which distinguishes a port that is not open from a message that is not arriving.

8.7 Procedure

The following procedure constructs the output path using the Switch TOP bank.

1. Add a Movie File In TOP for each asset to be made available, and match their resolution and format.
2. Add a Switch TOP and connect every Movie File In as an indexed input, in a fixed and recorded order.
3. Add an OSC In operator, set its Protocol to Messaging (UDP), and set its Network Port to the agreed port. The projector-output example used port 8000 at local address 192.168.50.20.
4. Isolate the incoming index with a Select operator, and apply it to the Switch TOP's Index parameter through a parameter expression, or apply it within the onReceiveOSC callback where a callback is preferred.
5. Where the sender transmits momentary triggers, convert them to a held value with a Trigger CHOP or a Count CHOP.
6. Route the Switch TOP through KantanMapper, where warping is required, and into the projector output window set to Perform mode.
7. Transmit indices across the network and confirm that the displayed asset changes accordingly.

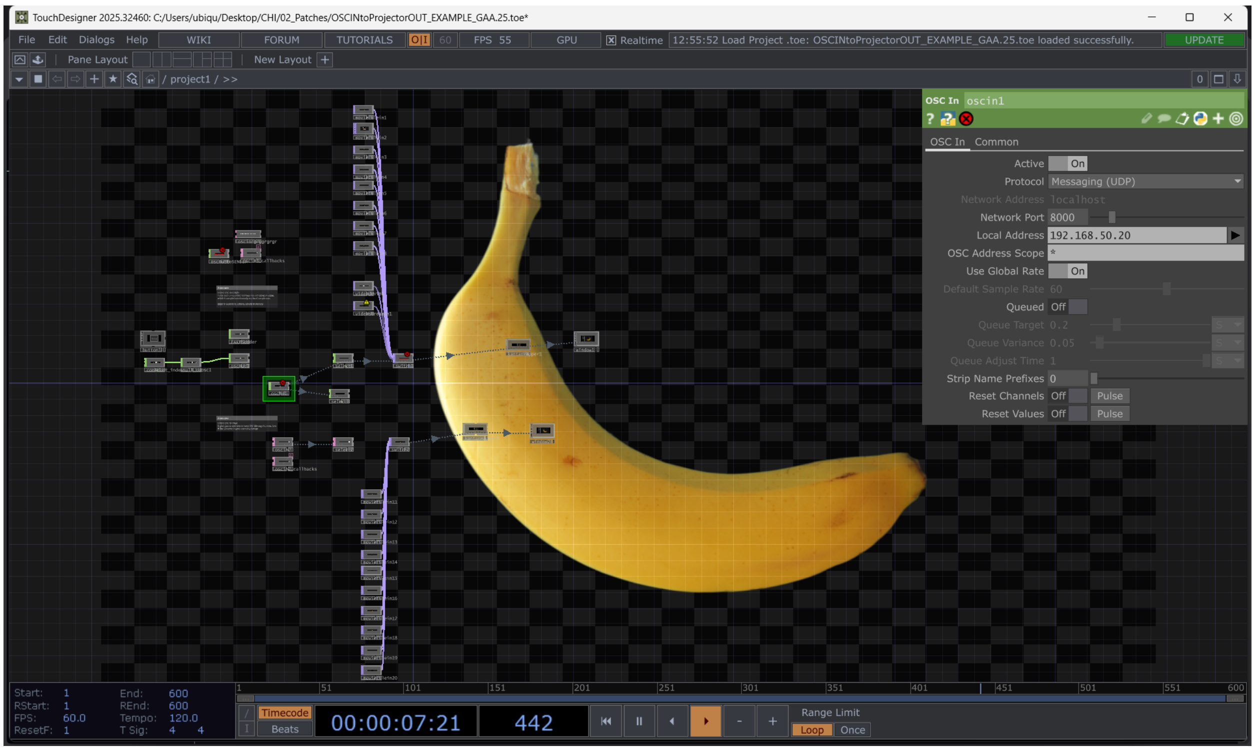

Figure 8. The OSCINtoProjectorOUT patch, a complete realized output path. An OSC In DAT listening on port 8000 at local address 192.168.50.20 selects an asset and routes it to a projector output window.

The index is meaningful only if the order of the sources remains constant. The order should be established once and recorded, so that a given index denotes the same asset on every machine. If the order is altered independently on one machine, the room and its digital model will diverge.

9. Input-Driven Interaction

An environment that responds to its occupants requires input. Prior to the construction of the complete sensing system, two interaction experiments were conducted to establish the control model, in which an input determines a state and the state determines the projection. The first employs a keyboard; the second employs camera-based hand tracking.

9.1 Keyboard input

The Keyboard In COMP captures key presses and exposes them within the patch, providing a means of switching between states in TouchDesigner. It was used to switch the output of a video projection by keypress. Although elementary, this experiment employs the same control structure as the OSC index: an input arrives, and the environment advances to a different state. The keyboard may be replaced by OSC, sensor, or engine input without alteration to the remainder of the patch.

1. Add a Keyboard In COMP to the patch.
2. Map the intended keys to the states to be triggered.
3. Route the resulting value to the parameter that selects the state, such as the Index of a Switch TOP.
4. Press a key and confirm that the projection changes. The keyboard may subsequently be replaced by any other input source without further change to the patch.

9.2 Hand tracking

A separate Python-to-OSC system developed by Presley Falkenberg was tested on the sensing side. The system uses Google Body Pose and OpenCV to track a hand through a camera and transmits the tracking data over OSC into TouchDesigner, where the hand movements selected among images and controlled the position of an object along the X and Y axes within a two-dimensional space. This applies the messaging pattern of Sections 6 through 8 to input generated by a human subject.

Keyboard input, an OSC index, and hand tracking are instances of a single control structure, in which an input produces a value and a value selects a state. Maintaining this structure, so that each input is expressed as OSC and each visual responds to a value, allows additional sensors to be incorporated without redesign of the system core.

10. Aesthetic Development

The preceding sections concern the technical infrastructure of the room. A portion of the fellowship was directed toward the visual content itself. The operators examined in this work include noise, displacement, lookup, and feedback, together with the relationships between audio and visual sources. Figure 9 shows one such study, a displacement network producing an organic color field. These studies were conducted as exercises in the tool's image-generation capabilities rather than as components of the delivery pipeline.

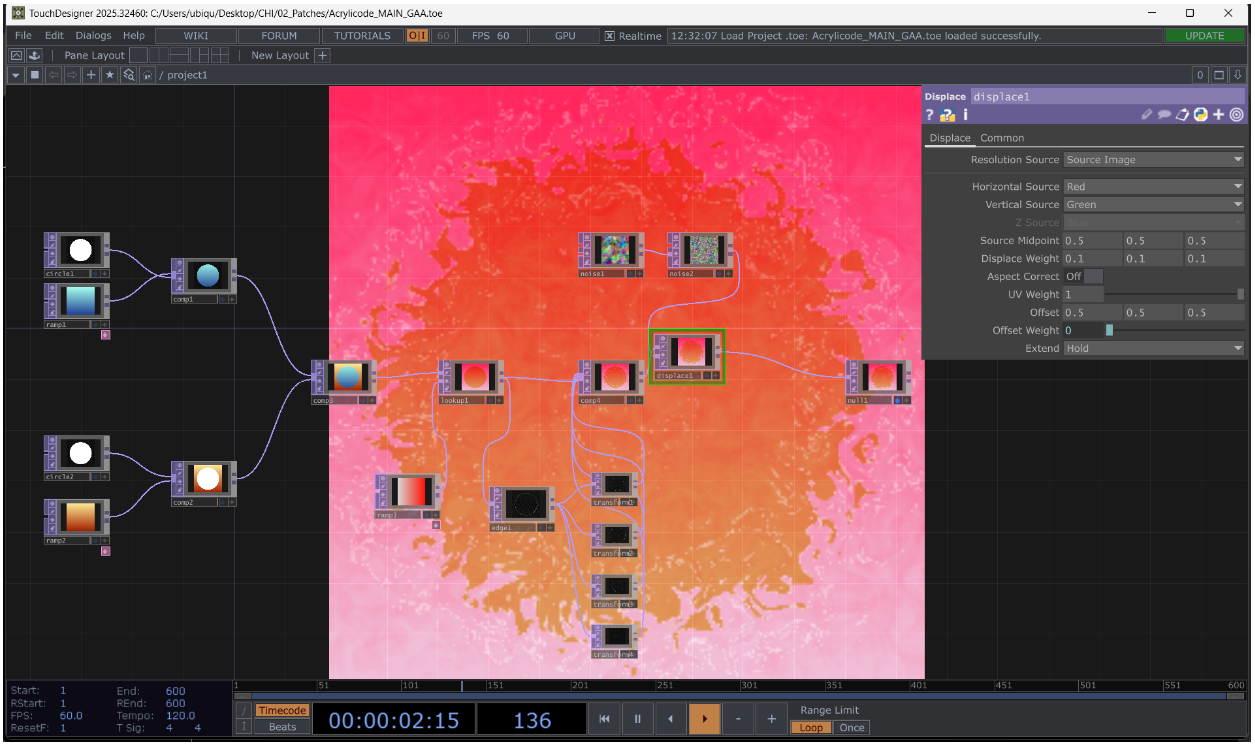

Figure 9. A displacement study (Acrylicode). Noise sources feed a Displace TOP through a lookup, producing an organic color field.

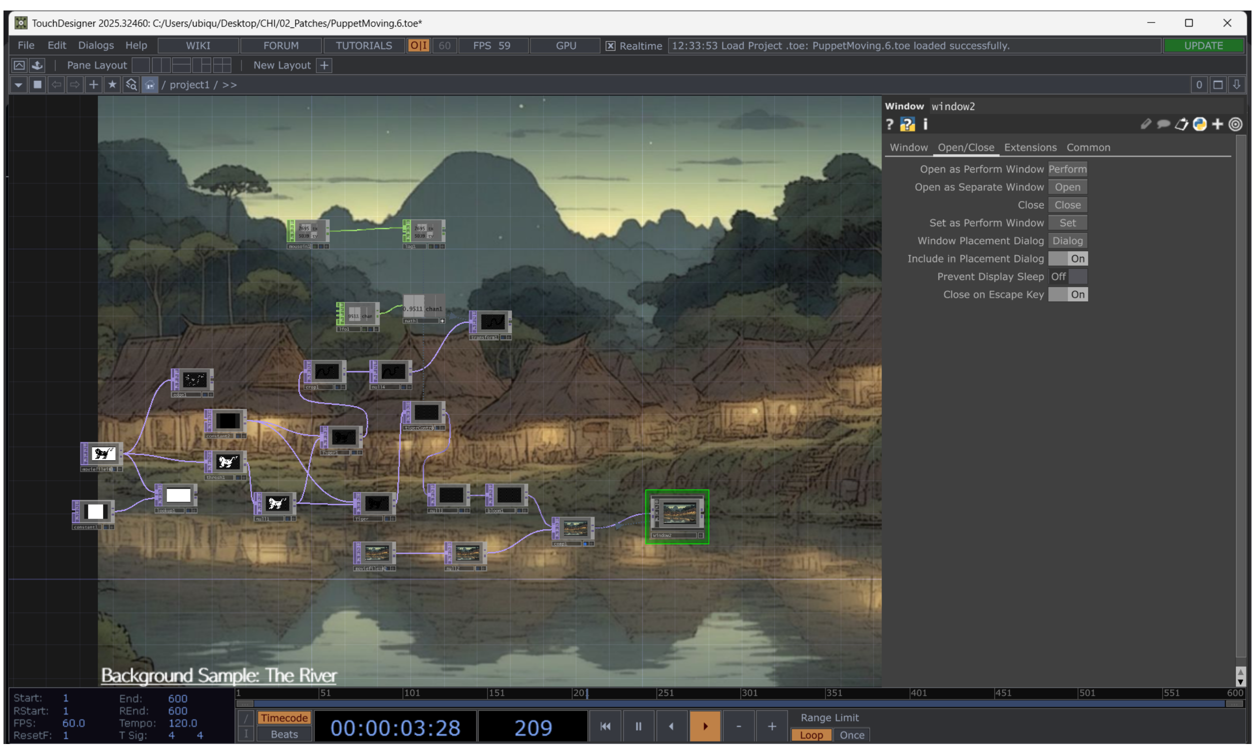

Figure 10. A compositing study (PuppetMoving). A layered scene is assembled toward an output window, examining movement and composition.

On one occasion the intended objective was the completion of a single larger work, and the result was instead several smaller studies. The latter outcome proved more useful, as each study permitted the examination of a distinct technique and increased familiarity with the environment. The relevant capability for the Self-Aware Room is the rapid translation of a concept into a visual result, and this capability is developed more efficiently through repeated small exercises than through the completion of individual finished works.

11. System Integration

The concluding objective of the fellowship was the operation of the individual systems as a single pipeline. The target architecture is stated simply: a controlling program determines a state, transmits it over OSC, and both the physical projection system and a digital model reflect that state simultaneously.

The configuration agreed upon with Sunima and Isaiah was a complete pipeline in which a Python program controlled the index of the video files, and that index was received by both the digital model and the live-room projector, causing both to display the same asset. The objective was not visual complexity but the demonstration that a single source of state, the Python program, could be honored simultaneously by each downstream system.

In the integrated test, the Python program transmitted OSC to both TouchDesigner and Unity across the local Ethernet network. Following the correction of the repository fault and the firewall and dependency issues described in Section 7, the pipeline functioned. The Python program was able to select the displayed image and to control parameters of a three-dimensional object within TouchDesigner, and the hand-tracking system operated through the same path. This established that the separate software environments of the room can function as a single coordinated system, which is the premise on which the Self-Aware Room depends.

12. Limitations

Several limitations apply to the systems documented in this report. The visual output of the final room depends on capabilities restricted to the Educational license tier, and the systems tested under the Non-Commercial tier will require that license before they can operate at the intended resolution. The integrated pipeline has been demonstrated but not hardened; it functioned in a controlled test and does not yet define behavior for missed or malformed messages, which a live performance would require. The stability of a multi-projector configuration depends on correct window configuration and on adequate graphics hardware, and configurations beyond approximately four projectors introduce the additional complexity of multiple graphics cards. Finally, several of the procedures depend on network conditions, including firewall configuration and correct addressing, that must be re-established in each new physical location. These limitations are addressable through appropriate licensing, further development, and documentation, and do not affect the validity of the methods described.

13. Future Work

Several directions extend the work documented here. The most immediate is the acquisition of the Educational license, which removes the resolution and TouchEngine limitations affecting the final installation. The integrated pipeline should be developed from a demonstrated configuration into a dependable system, including defined behavior in the event of interrupted communication. A determination should be made regarding the show-control layer, whether through the adoption of QLab by way of Syphon or TouchEngine, the adoption of a TouchDesigner-native cue system such as CuryCue, or the development of a dedicated application. The sensing layer should be extended beyond hand tracking, with each additional input expressed through OSC in order to preserve the existing system structure. The development of the room's aesthetic content should proceed in parallel with this technical work.

14. Conclusion

The fellowship established the technical foundation of the Self-Aware Room. The licensing requirements of the project were determined, projection mapping was made reproducible for single and multiple projectors, a messaging layer capable of operating across machines was constructed, input methods were demonstrated, and a complete pipeline was operated end to end with the participating systems in agreement. Each of these systems is documented above with an accompanying procedure. The work is not complete, and the development of the room into a dependable and aesthetically realized environment remains for subsequent fellows. This report is intended to render the established methods reproducible, so that the remaining work may proceed from a documented foundation.

Appendix A: Equipment and Cabling

The following terms are used in the systems diagram and the procedures throughout this report.

Item	Role in the configuration
Computer running TouchDesigner	The source of all media and the machine executing the patches.
EL-265F projectors (three in the documented rig)	Cover the shared projection surface, one mapped region each.
Optoma Short-Throw Pocket LED	Small-form-factor test projector; provides no zoom or corner adjustment.
Netgear network switch	Provides the dedicated local Ethernet network for the machines.
HDMI cable	Carries video from a native HDMI output.
HDMI-to-USB-C cable (two units)	Carry video from the computer's USB-C outputs to projectors.
USB-C to HDMI cable (magenta in the diagram)	Signal path from the computer to each projector.
IEC C13 power cord (red in the diagram)	Power to each projector.
Edison stinger (orange in the diagram)	Power to the computer's supply.

Appendix B: Configuration Reference

The following settings are referenced in the procedures above.

Context	Setting	Value or action
KantanMapper	Open as Separate Window	Engage to activate live output.
KantanMapper	Display (panel settings)	Disable so the editor is not projected.
Perform window	Borders	Off.
Perform window	Monitor	The target display (1 in the single-projector test).
Perform window	Opening Size	Fill.
Multi-projector	Window mode	Perform, not Working.
OSC In DAT	Protocol	Messaging (UDP).
OSC In DAT	Network Port	The listening port, for example 9000 or 8000.
OSC In DAT	Local Address	The machine's address, for example 192.168.50.20.
OSC In DAT	onReceiveOSC callback	Read the value and apply it to the target parameter.
Switch TOP	Index	Driven by the incoming OSC value to select a source.
Network	Firewall	Open the OSC port on every sending and receiving machine.
Network	Ping	Expect zero percent loss before launching OSC.
Optoma projector	Power	19V input.

Appendix C: Sources by Topic

Projection mapping with KantanMapper

- moha.wiki/Projection_Mapping_with_Touch_Designer_Using_KantanMapper
- derivative.ca/UserGuide/Palette:kantanMapper
- youtube.com/watch?v=mTH7ZB4x47Q

Multiple projectors and graphics cards

- derivative.ca/UserGuide/Using_Multiple_Graphic_Cards
- derivative.ca/UserGuide/Projection_Mapping

Show control and cueing

- github.com/netzzy/CuryCue
- github.com/netzzy/CuryCue/blob/main/README.md
- afonin.media/curiosity-opera
- qlab.app/docs/v4/video/video-surface-editor

Open Sound Control and messaging

- opensoundcontrol.stanford.edu/spec-1_0-examples.html
- hexler.net/protokol
- hexler.net/protokol/manual/introduction
- github.com/jonglissimo/td-osc-query-server

Licensing

- The TouchDesigner and TouchPlayer license comparison published by Derivative, reproduced in Section 2.

Project documentation and captures

- The Self-Aware Room repository (CHI-CityTech/Self-Aware-Room), including the tiered-subscription document.
- Process video and audio captures archived to the project Dropbox.

Prepared by Gabriel A. Aguilar for the CHI Fellowship, Self-Aware Room Initiative, New York City College of Technology. Faculty advisor Professor David Smith. Documented fellowship period July 15 to August 13, 2026.